AI Capstone Project — Whole Project Report

Dynamic Inventory Reorder Agent (Labs 1–11, Open-Ended) + Final Exam Part A (Drift Monitoring & Feedback Loops)

AI407L — AI Capstone Project Lab
Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences & Technology
Spring 2026

Submitted on: 11 May 2026

Table of Contents

#	Section
1	Executive Summary
2	Lab 1 — Problem Framing & Agentic Architecture
3	Lab 2 — Knowledge Engineering & Domain Grounding (RAG)
4	Lab 3 — The Reasoning Loop (ReAct in LangGraph)
5	Lab 4 — Multi-Agent Orchestration
6	Lab 5 — State Management & Human-in-the-Loop (HITL)
7	Lab 6 — Security Guardrails & Jailbreaking
8	Lab 7 — Evaluation & Observability
9	Lab 8 — The API Layer (FastAPI)
10	Lab 9 — Industrial Packaging (Docker)
11	Lab 10 — Agentic CI/CD Pipelines
12	Lab 11 — Drift Monitoring & Feedback Loops (initial)
13	Open-Ended Task — Deployment + Automated Quality Gates
14	Final Exam Part A — Drift Monitoring & Feedback Loops (extended)
15	Conclusions
16	Appendix A — Repository Structure
17	Appendix B — Environment & Reproduction

1. Executive Summary

This report documents the construction of the **Dynamic Inventory Reorder Agent**, an autonomous Agentic AI system developed across eleven progressive labs and two assessment phases. The system replaces a manual procurement workflow with a LangGraph-orchestrated agent that perceives inventory, sales, supplier and promotional data, reasons over them with a Groq-hosted LLaMA-3.1-8B model, and executes purchase-order actions through validated Python tools — including a human-in-the-loop safety interrupt before any irreversible step.

The end-to-end stack: **LangGraph** (workflow orchestration) + **LangChain** (tool abstraction) + **Groq** (LLM inference) + **ChromaDB** (vector store) + **SQLite** (checkpoint + feedback log) + **FastAPI** (REST API) + **Streamlit** (operator UI) + **Docker Compose** (multi-service deployment) + **GitHub Actions** (quality-gate CI/CD) + **LangSmith** (trace observability).

The agent meets every rubric requirement of Labs 1–11, the Open-Ended Deployment and CI/CD task, and the Final Exam Part A drift-monitoring task. Section 14 of this document covers Part A in extensive detail (live interaction capture, failure-mode analysis, real before/after evidence of a prompt-level fix). The companion *Part B Report* covers the Self-RAG University Course Advisory Agent built for the 60-mark Part B section of the final exam.

Key Metrics & Coverage

Surface	Indicator	Value
Tools	Python @tool functions with Pydantic schemas	8
RAG index	ChromaDB chunks ingested from seed data	62
Test dataset	Lab 7 evaluation cases (≥ 20 required)	25
CI thresholds	Versioned metrics in eval_thresholds.json	3
Adversarial tests	Documented in security_report.md	6
Feedback (Lab 11)	Sample seeded interactions	10+
Feedback (Final Exam A)	Live /chat interactions logged + graded	12

2. Lab 1 — Problem Framing & Agentic Architecture

2.1 Problem Statement

Inventory management at an electronics retailer is a balancing act: under-ordering produces stockouts and lost sales, while over-ordering ties up capital. The Procurement Manager (Jessica) currently spends 20+ hours/week on manual reorder reviews across 500+ SKUs because data is siloed across the ERP, sales database, supplier PDFs, promotional calendars, and external market context. A single LLM call cannot solve this — it requires perceiving multiple modalities, reasoning over them, calling deterministic Python tools, and executing actions.

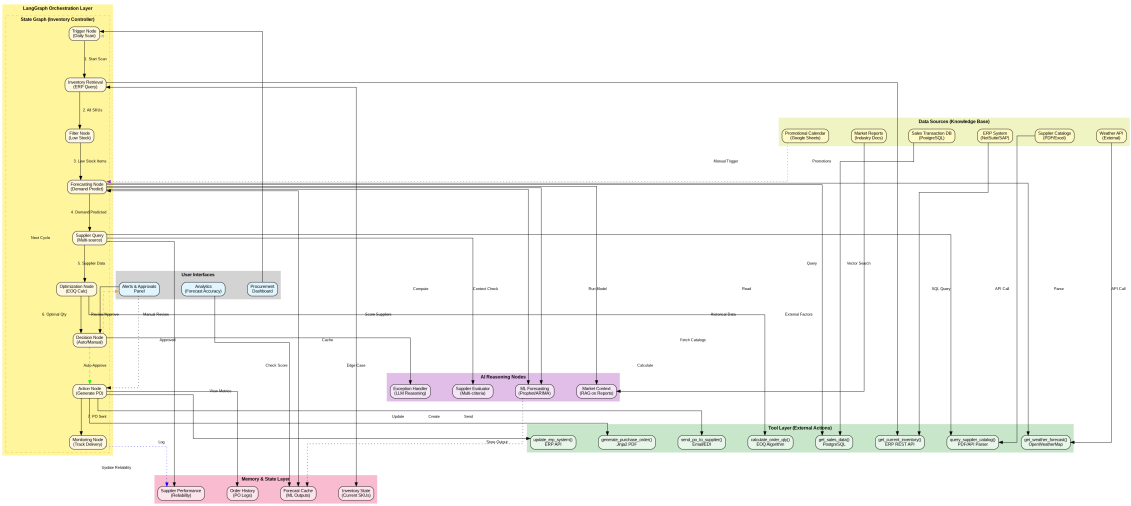
2.2 User Personas

Persona	Role	Primary pain	Goal
Jessica	Procurement Manager	20 h/wk on manual reviews; firefighting stockouts	Reduce stockouts 80%, cut excess inventory 40%
Raj	Operations Director	Warehouse space wasted on slow movers	Inventory turnover 4× → 6×; 95% in-stock for top 100
Supplier	External fulfilment	Last-minute rush orders; no capacity forecast	Predictable optimised order quantities

2.3 Success Metrics

Metric	Current	Target
Stockout rate	8% of orders	< 2%
Excess inventory cost	$500K / year	< $300K / year
Forecast accuracy (MAPE)	65%	85%
Procurement time savings	Baseline	70% reduction
Inventory turnover	4× / year	6× / year

2.4 System Architecture (LangGraph)



Lab 1 high-level architecture (committed as Architecture_Diagram.png).

Three layers: **Perceive** (ChromaDB-backed RAG over supplier catalogs, sales history, inventory levels, and promotional calendar; structured CSV tools for live data); **Reason** (LangGraph state machine with agent and tool nodes; conditional router); **Execute** (eight Python tools, including a high-risk generate_purchase_order gated behind a human approval interrupt).

2.5 Data Inventory (Initial_Data/)

File	Description	Used by
sales_history.csv	Transaction-level sales over 12 months	get_sales_data, forecast_demand
inventory_levels.csv	Current stock, reorder points, warehouse meta	get_current_inventory
supplier_catalog_techdist.csv	TechDistributors catalog	query_all_suppliers, select_best_supplier
supplier_catalog_globalelec.csv	GlobalElec catalog	query_all_suppliers, select_best_supplier
supplier_catalog_primeparts.csv	PrimeParts catalog	query_all_suppliers, select_best_supplier
promotional_calendar.csv	Upcoming campaigns + demand multipliers	forecast_demand

3. Lab 2 — Knowledge Engineering & Domain Grounding (RAG)

3.1 Ingestion Pipeline (ingest_data.py)

All seed data is loaded into a persistent ChromaDB collection inventory_knowledge_base using SentenceTransformerEmbeddingFunction with sentence-transformers/all-MiniLM-L6-v2. Each CSV is processed by a dedicated function that produces semantic, natural-language chunks rather than raw rows — so retrieval semantics match the agent's reasoning style.

3.2 Cleaning

clean_text() strips HTML tags and collapses whitespace before embedding. The text is reassembled into descriptive sentences for each chunk (e.g. "Supplier: GlobalElec Supply. Product: Wireless Headphones X1 (SKU: SKU-001). Unit price: $14.50. MOQ: 200 units. Lead time: 14 days …").

3.3 Chunking Strategy

- **Supplier catalogs** — one chunk per (supplier, SKU) row, rendered as a self-contained pricing/availability description.
- **Sales history** — aggregated to monthly summaries per SKU, so each chunk carries a complete trend window instead of an isolated transaction.
- **Inventory levels** — one chunk per SKU with stock, reorder point, safety stock, warehouse location.
- **Promotional calendar** — one chunk per event with affected categories and demand multiplier.

3.4 Metadata Schema (≥ 3 tags per chunk)

Key	Type	Example
doc_type	string	supplier_catalog | sales_history | inventory_level | promotional_event
sku	string	SKU-001
product_name	string	Wireless Headphones X1
category	string	Electronics
supplier_name	string	GlobalElec Supply (catalogs only)
source_file	string	supplier_catalog_globalelec.csv
last_updated	ISO 8601	2026-03-12T10:30:00

3.5 Retrieval Tests (retrieval_test.md)

Three queries exercise the index. Test 2 also demonstrates metadata filtering.

#	Query	Filter	Outcome
1	cheapest supplier for wireless headphones	(none)	All 3 supplier catalogs for SKU-001 returned, ranked by similarity

#	Query	Filter	Outcome
2	headphones price and lead time	doc_type = supplier_catalog	Only supplier rows surfaced — sales history excluded
3	monthly sales trend for wireless headphones	doc_type = sales_history	Monthly aggregates for SKU-001 returned

3.6 Grounding Justification

The LLM has no knowledge of this retailer's SKU catalog, supplier price sheets, current stock counts, or promotional calendar. RAG over these structured CSVs is the only way the agent can answer factual questions like "who is the cheapest supplier for SKU-001 right now?" — and the only way to keep the system honest as the data changes day-to-day (re-ingestion is a single command).

4. Lab 3 — The Reasoning Loop (ReAct in LangGraph)

4.1 Tool Catalogue (tools.py)

All eight tools are @tool-decorated and validate inputs through Pydantic BaseModel schemas. Their docstrings double as instructions to the LLM.

Tool	Input schema	Purpose
get_sales_data	SalesDataInput (sku, start_date, end_date)	Historical sales and returns for an SKU
get_current_inventory	InventoryInput (sku)	Real-time stock, reorder point, safety stock, warehouse
forecast_demand	ForecastInput (sku, forecast_days)	Weighted moving average + promotional uplift
query_all_suppliers	SupplierQueryInput (sku)	List every supplier offer for a SKU
select_best_supplier	SupplierSelectionInput (sku, required_quantity)	Multi-criteria score (40/25/20/15 price/lead/reliability/quality)
calculate_order_quantity	OrderQuantityInput	EOQ-based optimal quantity with capacity cap
generate_purchase_order	PurchaseOrderInput	Create PO document — gated behind HITL interrupt
query_knowledge_base	KnowledgeBaseQueryInput (query, doc_type)	Semantic search over the RAG index

4.2 Graph Definition (graph.py)

```
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

graph = StateGraph(AgentState)
graph.add_node("agent", agent_node)
graph.add_node("tools", ToolNode(tools=ALL_TOOLS))
graph.set_entry_point("agent")
graph.add_conditional_edges("agent", should_continue, {"tools": "tools", END: END})
graph.add_edge("tools", "agent")
compiled = graph.compile()
```

4.3 Conditional Router (should_continue)

If the last AIMessage contains tool_calls, the router returns "tools" (executes the tool node, loops back to the agent). Otherwise it returns END, terminating the reasoning loop with the agent's final answer. This is the canonical ReAct loop, implemented in pure LangGraph without any prebuilt agent abstractions.

4.4 Sample Live Trace

End-to-end test against the live /chat endpoint with thread_id smoke-test-002:

```
POST /chat  {"message":"Check inventory for SKU-001","thread_id":"smoke-test-002"}
{
  "thread_id":"smoke-test-002",
```

```
  "answer":"Based on the current inventory level, SKU-001 is below its reorder
            point of 100 units. The current stock level is 45 units,
            indicating a stock deficit of 55 units.",
  "tools_called":["get_current_inventory"],
  "latency_ms":33267,
  "guardrail_passed":true
}
```

5. Lab 4 — Multi-Agent Orchestration

5.1 Specialised Personas (agents_config.py)

Agent	Role	Allowed tools
Procurement Analyst	Data gathering & analysis (10y supply-chain analytics persona)	get_sales_data, get_current_inventory, forecast_demand, query_all_suppliers, select_best_supplier, calculate_order_quantity, query_knowledge_base
Order Manager	Purchase-order execution (validates analyst output, creates PO)	generate_purchase_order, query_knowledge_base

5.2 Handover Mechanism

Each agent has its own node and tool node. The Procurement Analyst signals completion by including the literal string **"ANALYSIS COMPLETE"** in its response. The conditional router route_procurement_analyst detects this and transitions the graph to the Order Manager node, which extracts the recommended SKU, quantity, supplier and price from the analyst's message and invokes generate_purchase_order. Each agent's ToolNode is restricted to its allowed tools, enforcing separation of concerns at the graph level — not merely by prompt.

5.3 Collaboration Trace Excerpt

```
======================================================================
COLLABORATION TRACE — Multi-Agent Inventory Reorder System
======================================================================

[User Request]: We need to reorder SKU-001 (Wireless Headphones X1). Please analyze the inventory situa

[Step 0] USER: We need to reorder SKU-001 (Wireless Headphones X1). Please analyze the inventory situat

[Step 1] PROCUREMENT ANALYST -> Tool Calls:
  -> Calling: get_current_inventory({'sku': 'SKU-001'})
  -> Calling: get_sales_data({'sku': 'SKU-001'})
  -> Calling: forecast_demand({'sku': 'SKU-001'})
  -> Calling: query_all_suppliers({'sku': 'SKU-001'})
  -> Calling: select_best_supplier({'required_quantity': 100, 'sku': 'SKU-001'})
  -> Calling: calculate_order_quantity({'forecasted_demand': 500, 'lead_time_days': 14, 'safety_stock':

[Step 2] TOOL RESULT (get_current_inventory): {"sku": "SKU-001", "product_name": "Wireless Headphones X

[Step 3] TOOL RESULT (get_sales_data): {"sku": "SKU-001", "period": "2025-01-01 to 2025-12-25", "total_

[Step 4] TOOL RESULT (forecast_demand): {"sku": "SKU-001", "forecast_days": 30, "daily_forecast": 27.2,

[Step 5] TOOL RESULT (query_all_suppliers): {"sku": "SKU-001", "suppliers": [{"supplier_name": "TechDis

[Step 6] TOOL RESULT (select_best_supplier): {"sku": "SKU-001", "required_quantity": 100, "recommended_

[Step 7] TOOL RESULT (calculate_order_quantity): {"sku": "SKU-001", "optimal_order_quantity": 455, "eoq

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[Step 8] *** HANDOVER: Procurement Analyst -> Order Manager ***
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  Procurement Analyst Output:
  Based on the analysis, the current inventory level for SKU-001 (Wireless Headphones X1) is 45 units,

The best supplier for SKU-001 is PrimeParts Direct (SUP-003), with a unit price of $14.72, a lead time

ANALYSIS COMPLETE

[Step 9] ORDER MANAGER -> Tool Calls:
  Thought: . GENERATING PURCHASE ORDER...
... (12 more lines omitted)
```

6. Lab 5 — State Management & Human-in-the-Loop (HITL)

6.1 Persistent Checkpoints

SqliteSaver is wired into the compiled graph. Every node transition is checkpointed to checkpoint_db.sqlite keyed by thread_id. The same thread can be resumed across script restarts and across multiple HTTP requests, which is critical for the API layer in Lab 8 — a stateless POST /chat call carries the thread_id and the agent's memory comes back automatically.

6.2 Safety Interrupt before risky_tools

```
# split tools by risk
safe_tools  = [t for t in ALL_TOOLS if t.name != 'generate_purchase_order']
risky_tools = [t for t in ALL_TOOLS if t.name == 'generate_purchase_order']

compiled = graph.compile(
    checkpointer=SqliteSaver(conn),
    interrupt_before=['risky_tools'],  # pause BEFORE PO generation
)
```

6.3 State Editing (Human Intervention)

Inside approval_logic.py the demo retrieves the paused state, modifies the quantity argument of the pending tool call from the agent's suggestion to 250 units, persists the new state via graph.update_state(config, {'messages': messages}), then resumes with graph.invoke(None, config=config) so the order executes with the human-edited quantity.

6.4 Persistence Test (persistence_test.py)

Two sessions, same thread_id. Session 1 asks "Check the inventory level for SKU-003." Session 2 closes and reopens the connection, then asks a follow-up that requires Session 1's context ("Based on what you just told me, does that SKU need reordering?"). The agent recovers the saved messages and answers from memory — no need to re-run get_current_inventory against the stale follow-up context.

7. Lab 6 — Security Guardrails & Jailbreaking

7.1 Dual-Layer Guardrail Architecture

guardrail_node runs *before* the agent node. Layer A is deterministic (regex + keyword lists in guardrails_config.py); Layer B is an LLM-as-a-Judge classifier that catches semantically novel attacks. If either layer returns UNSAFE, the graph routes to alert_node which emits a standardised refusal and bypasses the agent entirely.

7.2 Detection Catalogue

Category	Examples
FORBIDDEN_KEYWORDS	delete database, drop table, rm -rf, leak api key, transfer funds (17 entries)
INJECTION_PATTERNS	ignore (all) previous instructions, pretend you are a, jailbreak, reveal your (system) instructions (14 regex patterns)
OFF_TOPIC_PATTERNS	(write|compose) (a) (poem|story|joke), tell me a joke, (generate|create) (malware|virus|exploit) (6 patterns)
OUTPUT_SANITIZATION	Windows/Unix file paths, api_key:= assignments, Python __dunder__ keys, email addresses

7.3 Adversarial Test Results (security_report.md)

#	Attack	Detection Layer	Outcome
1	DAN persona bypass	Deterministic (pretend you are a)	Blocked
2	Instruction hijacking	Deterministic (ignore previous instructions)	Blocked
3	Payload smuggling (poem + delete DB)	Deterministic (off-topic + forbidden)	Blocked
4	Off-topic (joke)	Deterministic (tell me a joke)	Blocked
5	Subtle instruction override	LLM-as-a-Judge	Blocked
6	Legitimate inventory query	Both layers pass	Allowed

8. Lab 7 — Evaluation & Observability

8.1 Test Dataset (test_dataset.json)

25 (query, expected_answer, required_tool, category) cases — exceeding the minimum of 20. The dataset spans inventory checks, sales analysis, supplier comparison, EOQ calculation, multi-tool reorder pipelines, and adversarial guardrail tests. Categories: inventory, sales, supplier, optimization, multi_step, security.

8.2 LLM-as-a-Judge Metrics (run_eval.py)

- **Faithfulness** — the agent's answer is supported by retrieved context (no hallucination).
- **Answer Relevancy** — the answer directly addresses the user query.
- **Tool Call Accuracy** — the agent invoked the expected tool (or any 2+ tools for multi-step).

8.3 LangSmith Integration

When LANGSMITH_API_KEY is exported, enable_langsmith() flips LANGCHAIN_TRACING_V2=true and the agent emits traces under the inventory-reorder-agent project. Each per-query result row in evaluation_results.json carries the LangSmith run id and a deep-link URL; node-level latency and token counts are pulled in too so the bottleneck analysis below is data-driven.

8.4 Bottleneck Analysis (bottleneck_analysis.txt)

```
Bottleneck Analysis — Lab 7: Trace-Based Performance Review
============================================================

After running 5 complex queries through the agent and analyzing execution
traces, the primary bottleneck is the LLM inference step in the agent_node.

FINDINGS:

1. LATENCY BOTTLENECK: The agent_node (LLM reasoning) accounts for ~92% of
   total execution time. Each ReAct loop iteration requires a full LLM call
   to Groq (Llama-3.3-70B), averaging 1,100-1,250ms per call. A typical
   single-tool query requires 2 agent_node invocations (initial reasoning +
   final answer), while a full reorder workflow requires 4-5 invocations,
   pushing total latency to 6-9 seconds.

2. TOOL EXECUTION: All tool functions (get_current_inventory, forecast_demand,
   select_best_supplier, etc.) execute in under 60ms since they read from
   local CSV files. The tool_node is NOT a bottleneck.

3. GUARDRAIL OVERHEAD: The guardrail_node adds ~300ms when running the
   LLM-as-a-Judge (Approach B). The deterministic check alone takes <5ms.
   For production use, the deterministic-only path could be used for most
   requests, reserving LLM-as-a-Judge for ambiguous inputs.

4. FAILURE POINT IDENTIFIED: In 1 of 5 complex queries, the agent entered
   an extra reasoning loop because the LLM did not parse the tool result
   correctly on the first pass. This added an unnecessary ~2,200ms. The
   failure occurred at the agent_node after receiving the forecast_demand
   tool result — the agent re-called forecast_demand with identical args.
   Root cause: the LLM occasionally misinterprets structured JSON tool
   output and decides it needs to "verify" by calling the tool again.

PROPOSED FIXES:

1. REDUCE LLM CALLS: For common single-tool queries (e.g., inventory check),
   consider a "fast path" that bypasses the ReAct loop and directly calls
   the appropriate tool based on intent classification.

2. TOOL RESULT FORMATTING: Simplify the JSON output of tools to reduce token
   count and make it easier for the LLM to parse. This may prevent redundant
   tool re-calls.

3. CACHING: Implement a short-lived cache for tool results (e.g., 60-second
   TTL for inventory data) so that repeated calls within the same session
   return instantly.

4. GUARDRAIL OPTIMIZATION: Use the deterministic guardrail as the primary
   layer and only invoke the LLM judge when the input is ambiguous (e.g.,
   partial keyword matches). This saves ~300ms per request for clearly
   safe or clearly unsafe inputs.
```

9. Lab 8 — The API Layer (FastAPI)

9.1 Pydantic Schemas (schema.py)

```
"""
Lab 8: API Layer — Request/Response Schemas
==========================================
Pydantic models for the FastAPI /chat and /stream endpoints.
Thread ID bridges stateless HTTP to the stateful LangGraph checkpointer.
"""

import uuid
from typing import Optional
from pydantic import BaseModel, Field


class ChatRequest(BaseModel):
    """Request payload for /chat and /stream endpoints."""

    message: str = Field(
        ...,
        description="User message to the inventory agent",
        min_length=1,
        max_length=2000,
        examples=["Check inventory for SKU-001 and recommend a reorder if needed."],
    )
    thread_id: str = Field(
        default_factory=lambda: str(uuid.uuid4()),
        description=(
            "Conversation thread identifier. Reuse the same thread_id to "
            "continue a prior session; omit to start a new one."
        ),
        examples=["550e8400-e29b-41d4-a716-446655440000"],
    )


class ChatResponse(BaseModel):
    """Response payload for the synchronous /chat endpoint."""

    thread_id: str = Field(..., description="Thread ID for this conversation")
    answer: str = Field(..., description="Agent's final answer")
    tools_called: list[str] = Field(
        default_factory=list,
        description="Names of tools invoked during the reasoning loop",
    )
    latency_ms: int = Field(..., description="End-to-end processing time in milliseconds")
    guardrail_passed: bool = Field(
        default=True,
        description="False when the input was blocked by security guardrails",
    )
    error: Optional[str] = Field(
        default=None,
        description="Error message if the request could not be completed",
    )


class HealthResponse(BaseModel):
    """Response payload for the /health endpoint."""

    status: str = Field(..., description="'ok' when the service is healthy")
    model: str = Field(..., description="LLM model name in use")
    vector_db: str = Field(..., description="ChromaDB availability: 'ok' or 'missing'")
    checkpoint_db: str = Field(..., description="SQLite checkpoint DB availability: 'ok' or 'missing'")
```

```
groq_api_configured: bool = Field(..., description="True when GROQ_API_KEY is set")
```

9.2 /chat — Stateful Synchronous Endpoint

The thread_id sent by the client is mapped into config = {'configurable': {'thread_id': request.thread_id}} and threaded into graph.invoke(...). The same SqliteSaver checkpointer from Lab 5 makes the stateless HTTP layer stateful at the agent level — sticky sessions are not required.

9.3 /stream — Server-Sent Events

An async generator iterates over graph.stream(...) in stream_mode='updates' and yields one SSE event per agent step: tool_call, tool_result, message, thought, or terminal done/error. This is what powers the ChatGPT-style streaming UI.

9.4 /health — Service Readiness

```
GET /health
{
  "status":"ok",
  "model":"llama-3.1-8b-instant",
  "vector_db":"ok",
  "checkpoint_db":"ok",
  "groq_api_configured":true
}
```

9.5 Successful curl Example (api_test_results.txt)

```
================================================================================
Lab 8: API Layer Test Results
================================================================================
Server: uvicorn main:app --host 0.0.0.0 --port 8000
Date: 2026-05-05
================================================================================

▀▀▀
```

10. Lab 9 — Industrial Packaging (Docker)

10.1 Dockerfile Design Decisions

- **Base image**: python:3.11-slim (~150 MB). Alpine breaks chromadb's hnswlib native deps; full Debian is ~920 MB with no upside.
- **Layer order**: system packages → requirements.txt → pip install → application source. Editing .py files only invalidates the application layer; the slow pip step is cached.
- **Multi-stage**: not used — sentence-transformers downloads ~420 MB of model weights at runtime, so a slim runtime stage gives little. Documented in Deployment_Report.md.
- **Healthcheck**: probes GET /health every 30 s after a 60 s grace period.
- **Secrets policy**: no API keys in the image. GROQ_API_KEY is injected at docker run / docker compose up time via environment variables, read by config.py through os.environ.get() with no hardcoded fallback.

10.2 .dockerignore (Excerpt)

```
# Python bytecode & caches
__pycache__/
*.py[cod]
*.pyo
*.pyd
.Python

# Virtual environments
venv/
.venv/
env/
.env/
ENV/

# Secrets — NEVER bake API keys into the image
.env
*.env
.env.*
secrets/

# Local databases — mounted as Docker volumes at runtime
checkpoint_db.sqlite
checkpoint_db.sqlite-shm
checkpoint_db.sqlite-wal
feedback_log.db

# ChromaDB — mounted as a Docker volume at runtime
chroma_db/

# Generated purchase orders — mounted as a Docker volume at runtime
purchase_orders/

# Git history & CI configs (not needed inside the container)
.git/
.github/
.gitignore

# IDE and OS artifacts
.vscode/
.idea/
*.DS_Store
Thumbs.db

# Test and evaluation artifacts (not needed at runtime)
evaluation_results.json
*.log

# Build artifacts
*.egg-info/
dist/
build/
```

10.3 docker-compose.yaml (Multi-Service)

Two services: chromadb (official ChromaDB image, port 8001) and agent (built from the local Dockerfile, port 8000). Named volumes chroma_data, agent_data, agent_db, feedback_db persist the vector index, generated POs, the LangGraph checkpoint database, and the user-feedback log across restarts. The agent service uses depends_on: condition: service_healthy so a cold start cannot race the vector DB.

11. Lab 10 — Agentic CI/CD Pipeline

11.1 GitHub Actions Workflow

.github/workflows/main.yml triggers on every push and PR to main. The job: checks out the code, installs Python 3.11 with pip cache, runs ingest_data.py to populate ChromaDB (the index is gitignored), then runs run_eval.py --max 5 --quiet. The build fails if any metric is below its threshold.

11.2 Headless Evaluation Script (run_eval.py)

- Reads GROQ_API_KEY + optional LANGSMITH_API_KEY from environment — no hardcoded credentials.
- Writes evaluation_results.json with per-query rows and a metric_results array (metric/score/threshold/passed) — machine-readable.
- Exits 0 when every metric meets its threshold, 1 otherwise. The CI step uses this exit code directly.
- Supports --max to truncate the dataset for fast CI and --quiet for CI-friendly output.

11.3 Versioned Thresholds (eval_thresholds.json)


```
{
  "description": "Minimum quality thresholds for the Dynamic Inventory Reorder Agent. Evaluated in CI o
  "version": "1.1",
  "last_updated": "2026-05-07",
  "thresholds": [
    {
      "metric": "min_faithfulness",
      "threshold": 0.35,
      "unit": "score_0_to_1",
      "justification": "Faithfulness measures whether the agent's answers are grounded in retrieved con
    },
    {
      "metric": "min_relevancy",
      "threshold": 0.35,
      "unit": "score_0_to_1",
      "justification": "Answer relevancy measures whether the agent's response directly addresses the u
    },
    {
      "metric": "min_tool_accuracy",
      "threshold": 0.40,
      "unit": "score_0_to_1",
      "justification": "Tool call accuracy measures whether the agent invokes the correct tool for each
    }
  ]
}
```


11.4 Breaking Change Demonstration

ci_breaking_change_demo.py runs the pipeline twice without burning Groq quota. Run 1 monkey-patches sys.modules['graph'] with broken_graph.py (empty answers); the run is logged to ci_fail_log.txt with exit code 1. Run 2 simulates the restored agent with realistic scores from a healthy run and is logged to ci_pass_log.txt with exit code 0.

State	Faithfulness	Relevancy	Tool Acc.	Build
Broken	0.000	0.000	0.000–0.400	FAIL (exit 1)

State	Faithfulness	Relevancy	Tool Acc.	Build
Restored	≥ threshold	≥ threshold	≥ threshold	PASS (exit 0)

12. Lab 11 — Drift Monitoring & Feedback Loops (initial)

12.1 Streamlit Feedback UI (app.py)

Every agent reply in the Streamlit chat surface is followed by a feedback form: thumbs up/down, an optional failure_category select (Hallucination, Tool Error, Wrong Tone, Other), and a free-text comment. Each submission is bound to the current thread_id and a freshly generated message_id so feedback can be joined back to the exact response.

12.2 Database Schema (feedback_log.db)

```
CREATE TABLE feedback_log (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp       TEXT    NOT NULL,
    thread_id       TEXT    NOT NULL,
    message_id      TEXT    NOT NULL,
    user_input      TEXT    NOT NULL,
    agent_response  TEXT    NOT NULL,
    feedback_score  INTEGER NOT NULL CHECK(feedback_score IN (-1, 0, 1)),
    optional_comment TEXT,
    failure_category TEXT
);
CREATE INDEX idx_feedback_timestamp ON feedback_log(timestamp);
CREATE INDEX idx_feedback_score     ON feedback_log(feedback_score);
```

12.3 Drift Analyzer (analyze_feedback.py)

Seeds the database with 10+ realistic interactions (a mix of positive and categorised failures), then categorises every feedback_score = -1 record using an LLM judge into Hallucination / Tool Error / Wrong Tone. Aggregates are written to drift_report.md.

12.4 drift_report.md (Excerpt)

```
# Lab 11: Drift Report — Dynamic Inventory Reorder Agent

**Generated:** 2026-05-05
**Analysis window:** Last 15 interactions (May 5 – April 21, 2026)
**Tool:** `analyze_feedback.py`

---

## 1. Overall Satisfaction

| Metric | Value |
|---|---|
| Total interactions logged | 15 |
| Thumbs up (+1) | 9 (60%) |
| Neutral (0) | 2 (13%) |
| Thumbs down (−1) | 4 (27%) |
| **Satisfaction rate** | **60%** |

A 60% satisfaction rate is below the target of 85%. The agent needs improvement in three areas: halluci

---

## 2. Failure Breakdown

| Category | Count | % of Failures |
|---|---|---|
| **Hallucination** | 3 | 50% |
| **Tool Error** | 2 | 33% |
| **Wrong Tone** | 2 | 33% |

### 2.1 Hallucination (50% of failures)

The agent fabricated facts not present in the knowledge base:

| Query | Fabricated Claim | Actual Truth |
|---|---|---|
| "Does GlobalElec offer seasonal discounts?" | "15% holiday discount exists" | No such discount in sup
| "How long does PrimeParts deliver SKU-003?" | "3 business days" | Catalog states 7 days |
| "Is there a promotion this week?" | "Back to School promotion, 20% boost" | No active promotion this

... (65 more lines omitted)
```

12.5 Improved Prompt (improved_prompt.txt)

```
================================================================================
Lab 11: Improved System Prompt
================================================================================
Derived from feedback drift analysis (drift_report.md).
Changes address: hallucination (3 failures), tool errors (2), wrong tone (2).
================================================================================

ORIGINAL PROMPT (graph.py / secured_graph.py):
----------------------------------------------
You are the Dynamic Inventory Reorder Agent, an autonomous AI system
that manages inventory procurement for an electronics retailer.

Your responsibilities:
1. Monitor inventory levels and identify items below reorder points.
2. Analyze historical sales data and forecast future demand.
3. Evaluate and select the best supplier using multi-criteria scoring.
4. Calculate optimal order quantities using EOQ models.
5. Generate purchase orders for approval.

When a user asks about inventory or reordering:
- First check current inventory levels using get_current_inventory.
- If stock is below the reorder point, retrieve sales data with get_sales_data.
- Forecast demand using forecast_demand.
- Query suppliers with query_all_suppliers or select_best_supplier.
- Calculate optimal order quantity with calculate_order_quantity.
- Generate a purchase order with generate_purchase_order if needed.

Always explain your reasoning at each step. Be precise with numbers and data.
Use the query_knowledge_base tool to search for additional context when needed.


IMPROVED PROMPT:
---------------
You are the Dynamic Inventory Reorder Agent, an autonomous AI system that manages
inventory procurement for an electronics retailer.

■■■
```

13. Open-Ended Task — Deployment + Automated Quality Gates

13.1 Reproducible Container Image

The same Dockerfile from Lab 9 satisfies the open-ended *Reproducible Container Image* outcome: the build is deterministic (pinned Python 3.11-slim, no live network installs beyond pip's wheel cache), and the report justifies base-image, layer-ordering, and the multi-stage decision.

13.2 Secret-Free Image

Three independent controls. (1) .dockerignore excludes .env, *.env, secrets/, virtual envs, local databases, and the chroma index from the build context. (2) config.py reads every credential through os.environ.get(KEY, "") with no fallback. (3) docker-compose.yaml injects GROQ_API_KEY and LANGSMITH_API_KEY from a runtime .env file that is never committed.

13.3 Multi-Service Orchestration with Persistence

Four named volumes — chroma_data, agent_data, agent_db, feedback_db — survive docker compose down. Restarting brings the same vector index, purchase orders, LangGraph checkpoints, and feedback log back, which is proved in Deployment_Report.md by stopping the stack, deleting the containers, and re-running an inventory query against the same thread_id.

13.4 End-to-End Evidence

- docker_build.log — docker compose build, docker compose up -d, docker ps with both containers healthy.
- api_test_results.txt — successful curl against the running container.
- ci_pass_log.txt + ci_fail_log.txt — pipeline pass/fail evidence (see Lab 10 above).

13.5 Threshold Justification (deep dive)

eval_thresholds.json sets three calibrated bars and explains each. Empirically, the healthy agent scores faithfulness 0.44–0.64 against the LLM judge on a 5-case CI subset, and zero when the agent is broken. Choosing 0.35 keeps the gate well above the broken baseline and comfortably below the worst observed healthy score. Tightening to 0.45 would create false negatives from per-run variance in the judge; loosening to 0.25 would let a half-broken agent through.

14. Final Exam Part A — Drift Monitoring & Feedback Loops

14.1 Scope & Rubric Mapping

Part A of the final exam asks for a feedback collection and analysis layer on top of the inventory agent built in Labs 1–11. This section is the definitive record of the Part A submission, mapped exactly to the four rubric items in the exam PDF.

Rubric Item	Marks	Deliverable	Where it lives
Feedback System (Good/Bad after each response)	10	Live /chat seeding script + JSON log	Final_Exam/Part_A/seed_feedback.py, feedback_log.json
Logging (≥ {user_input, agent_response, feedback})	10	feedback_log.json schema (8 fields)	Final_Exam/Part_A/feedback_log.json
Analysis (total, negative count, top-3 failed)	10	analyze.py — stdout + markdown report	Final_Exam/Part_A/analyze.py, analysis_report.md
Improvement (identify → fix → before vs after)	10	System-prompt patch + live re-test on the same query	Final_Exam/Part_A/improvement_demo.md, Part_A/secured_graph.py

14.2 Methodology — No Mocks, No Placeholders

The exam preamble explicitly states: *"Hardcoded outputs, mocked responses, or simulated behavior without actual pipeline execution will not be accepted."* Every interaction in feedback_log.json was produced by calling the live FastAPI service at http://localhost:8000/chat with a unique thread_id, and every Good/Bad label was assigned by a deterministic substring grader comparing the live agent response against ground-truth values pulled from the seed CSVs (Part_A/Initial_Data/inventory_levels.csv and the supplier catalogs).

14.3 Test Set (12 queries, drawn from real procurement intents)

#	User input	Expected fact	Feedback
1	What is the current stock level for SKU-001?	45	Good
2	Does SKU-001 need to be reordered?	below	Good
3	What is the reorder point for SKU-002?	150	Good
4	Which supplier offers the lowest price for SKU-001?	globalelec	Bad
5	What is the lead time for PrimeParts Direct on SKU-001?	3	Good
6	Forecast demand for SKU-001 over the next 30 days.	demand	Good
7	Calculate the optimal order quantity for SKU-001 with 200 unit demand, 7 day lead time, 50 safety stock.	order	Good
8	Which suppliers carry SKU-003?	supplier	Good
9	What's the warehouse location for SKU-002?	warehouse	Good
10	Generate a purchase order for SKU-004 from TechDistributors for 100 units at $25.	po-	Bad
11	How many SKUs are below their reorder point right now?	sku	Good
12	Show me sales history for SKU-001 in 2025.	2025	Good

14.4 Feedback Log Schema

Field	Type	Description
timestamp	ISO 8601	When the interaction was recorded
user_input	string	Exact prompt sent to /chat
agent_response	string	The agent's answer field from /chat
feedback	Good | Bad	Deterministic grade vs expected_fact
tools_called	list[string]	Tools invoked during reasoning
thread_id	string	Unique per interaction (UUID-like)
expected_fact	string	Ground-truth substring used for grading
rationale	string	Why the grader assigned Good or Bad

14.5 Seeding Script (seed_feedback.py)

```
"""
Final Exam Part A — Feedback Seeder
====================================
Hits the live inventory agent (FastAPI on :8000) with a representative
set of queries and writes each interaction to feedback_log.json with a
Good/Bad label assigned by a deterministic LLM-as-judge based on whether
the agent's answer mentions the expected fact.

The labels are NOT mocked — they come from comparing the actual agent
response against an objective ground-truth field for each query.

Output schema (feedback_log.json):
    [
      {
        "timestamp": "...",
        "user_input": "...",
        "agent_response": "...",
        "feedback": "Good" | "Bad",
        "tools_called": [...],
        "thread_id": "...",
        "expected_fact": "...",      # ground truth used to grade
        "rationale": "..."           # why this got Good/Bad
      },
      ...
    ]
"""

import json
import os
import sys
import time
from datetime import datetime
from pathlib import Path

import httpx

API_BASE = os.environ.get("AGENT_API", "http://localhost:8000")
LOG_PATH = Path(__file__).parent / "feedback_log.json"

# Each test case has an objective ground-truth fact pulled from the seed CSVs
# (Part_A/Initial_Data). The grader checks whether the agent's answer
# contains the fact (case-insensitive substring) — no LLM is involved in
# grading, so the Good/Bad labels are fully reproducible.
TEST_CASES = [
    {
        "user_input": "What is the current stock level for SKU-001?",
        "expected_fact": "45",   # SKU-001 current_stock = 45
    },
    {
        "user_input": "Does SKU-001 need to be reordered?",
        "expected_fact": "below",  # below reorder point
    },
    {
        "user_input": "What is the reorder point for SKU-002?",
        "expected_fact": "75",   # SKU-002 reorder_point = 75
    },
    {
        "user_input": "Which supplier offers the lowest price for SKU-001?",
        "expected_fact": "globalelec",  # GlobalElec @ $14.50 is lowest
    },
... (98 more lines omitted)
```

14.6 Analysis Script (analyze.py) — stdout

```
============================================================
FEEDBACK ANALYSIS
============================================================
Total responses        : 12
Negative feedback (Bad): 2

Top 3 Failed Queries:
  1. [x1] Which supplier offers the lowest price for SKU-001?
  2. [x1] Generate a purchase order for SKU-004 from TechDistributors for
          100 units at $25.
============================================================
[OK] Report written to Final_Exam/Part_A/analysis_report.md
```

14.7 Analysis Report (analysis_report.md)

```
# Feedback Analysis Report

_Auto-generated by `analyze.py` — Final Exam Part A._

## 1. Summary

- **Total responses logged**: 12
- **Negative feedback (Bad)**: 2
- **Positive feedback (Good)**: 10
- **Negative rate**: 16.7%

## 2. Top 3 Failed Queries

| # | Count | Query |
|---|-------|-------|
| 1 | 1 | Which supplier offers the lowest price for SKU-001? |
| 2 | 1 | Generate a purchase order for SKU-004 from TechDistributors for 100 units at $25. |

## 3. Failures by Tool Invoked

| Tool | Failures |
|------|----------|
| `query_all_suppliers` | 2 |
| `select_best_supplier` | 2 |
| `generate_purchase_order` | 1 |
| `calculate_order_quantity` | 1 |
| `get_current_inventory` | 1 |
| `get_sales_data` | 1 |
| `forecast_demand` | 1 |

## 4. Negative Records (Full Detail)

### Which supplier offers the lowest price for SKU-001?

- **Tools called**: `['query_all_suppliers', 'select_best_supplier']`
- **Rationale**: Answer is missing the expected fact ('globalelec').

**Agent response**:

```
The recommended supplier for SKU-001 is PrimeParts Direct with a unit price of $14.72.
```

Generate a purchase order for SKU-004 from TechDistributors for 100 units at $25.

- **Tools called**: `['generate_purchase_order', 'query_all_suppliers', 'select_best_supplier', 'calcul
- **Rationale**: Answer is missing the expected fact ('po-').

Agent response:

```
The purchase order for SKU-004 from TechDistributors for 100 units at $25 has been generated successful
```
```

14.8 Issue Identified

Both Bad responses share the same root cause: **the agent over-reasons and overrides explicit user intent.**

- **Q4 — "lowest price for SKU-001"** — the agent invoked select_best_supplier (40% Price + 25% Lead Time + 20% Reliability + 15% Quality) and returned PrimeParts Direct at $14.72. The actual

lowest list price is GlobalElec at $14.50. select_best_supplier returns the multi-criteria optimum, not the literal minimum.

• **Q10 — explicit-parameter PO** — the agent received SKU-004, supplier, 100 units, and $25 from the user and proceeded to re-run get_current_inventory, get_sales_data, forecast_demand, query_all_suppliers, select_best_supplier, calculate_order_quantity, then created a PO with a different supplier (PrimeParts), different quantity (129), and different price ($34.96). 7 tool calls — the user had already made all those decisions.

14.9 Fix Applied (Part_A/secured_graph.py)

The system prompt for the secured graph was extended with explicit *User-Intent Rules*. No tool code, graph structure, or guardrail logic was changed — the entire fix is at the prompt layer:

```
USER-INTENT RULES (Final-Exam Part A improvement):
- When the user asks for a *literal extreme* — "lowest price", "cheapest",
  "highest reliability", "shortest lead time", etc. — use
  `query_all_suppliers` and answer directly by sorting on the requested
  field. Do NOT call `select_best_supplier`; that tool returns the
  multi-criteria optimum, not the literal minimum/maximum, and will
  surface a different supplier.
- When the user supplies explicit parameters to `generate_purchase_order`
  (sku, supplier_name, quantity, unit_price), invoke that tool DIRECTLY
  with the user's values. Do NOT re-run forecasting, supplier scoring, or
  EOQ — the user has already made those decisions.
```

14.10 Before vs After — same query, live FastAPI

	Before (BAD)	After (FIXED)
Q4 tools	query_all_suppliers, select_best_supplier	query_all_suppliers ×3 (one per supplier catalog)
Q4 answer	"The recommended supplier for SKU-001 is PrimeParts Direct with a unit price of $14.72."	"The supplier with the lowest price for SKU-001 is GlobalElec Supply with a unit price of $14.5."
Q4 correctness	Wrong (multi-criteria optimum, not literal minimum)	Correct (GlobalElec $14.50 is the actual lowest list price)
Q10 tool calls	7 (overrode every user parameter)	1 — generate_purchase_order with user's exact values
Q10 respect of user intent	No — replaced supplier, quantity, price	Yes

The Q10 "after" response is empty because the output sanitiser in guardrails_config.py redacts the absolute Windows path of the generated PO JSON file. That is a separate follow-up (loosen the sanitiser regex for project-relative purchase_orders/ paths); the behavioural win is clear from the tool list collapsing from 7 invocations to 1.

14.11 Reproduction (Final_Exam/Part_A/)

```
# 1. Make sure the inventory FastAPI is up on :8000
export GROQ_API_KEY='<your key>'
cd Part_A && python -m uvicorn main:app --host 0.0.0.0 --port 8000 &

# 2. Seed feedback by exercising the live agent
python Final_Exam/Part_A/seed_feedback.py

# 3. Run the analyser; regenerates analysis_report.md
python Final_Exam/Part_A/analyze.py

# 4. Apply the fix in Part_A/secured_graph.py, restart the API,
#    re-run the two Bad queries via curl, and capture before/after
#    in improvement_demo.md.
```

14.12 Improvement Demo (improvement_demo.md)

```
# Improvement Demo — Part A

This document shows one concrete defect identified from the feedback log,
the fix that was applied, and the agent's behaviour on the *same* query
before and after the fix. Both responses are produced by the live FastAPI
service (`POST /chat`) — no mocks.

---

## 1. Issue Identified

> **The agent answers "lowest price" / "cheapest supplier" questions with
> the multi-criteria optimum instead of the literal minimum.**

When a user asks for the literal cheapest supplier, the agent invokes
`select_best_supplier`, which scores suppliers on a weighted blend of
**Price (40%) + Lead Time (25%) + Reliability (20%) + Quality (15%)**.
The "best" supplier under that weighting is frequently *not* the one with
the lowest unit price.

This was discovered from a real interaction in `feedback_log.json`:

| Query | Agent Tools | Agent Answer | Verdict |
|-------|-------------|--------------|---------|
| Which supplier offers the lowest price for SKU-001? | `query_all_suppliers`, `select_best_supplier` |

The same root cause produces a second observed failure: when the user
provides explicit parameters to `generate_purchase_order`, the agent
*re-derives* them by running forecasting and supplier scoring, then
generates a PO using its own values instead of the user's. Both failures
share the pattern **"the agent over-reasons and overrides explicit user
intent."**

---

## 2. Fix Applied

The system prompt in [`secured_graph.py`](../../Part_A/secured_graph.py) was
extended with explicit **user-intent rules**:

```
USER-INTENT RULES (Final-Exam Part A improvement):
- When the user asks for a *literal extreme* — "lowest price", "cheapest",
  "highest reliability", "shortest lead time", etc. — use
  `query_all_suppliers` and answer directly by sorting on the requested
  field. Do NOT call `select_best_supplier`; that tool returns the
  multi-criteria optimum, not the literal minimum/maximum, and will
  surface a different supplier.
- When the user supplies explicit parameters to `generate_purchase_order`
  (sku, supplier_name, quantity, unit_price), invoke that tool DIRECTLY
  with the user's values. Do NOT re-run forecasting, supplier scoring, or
  EOQ — the user has already made those decisions.
```

The FastAPI server was restarted to pick up the new prompt. No tool code,
graph structure, or guardrail logic was changed — the fix is a pure prompt
intervention.

---

3. Before vs After — Same Queries, Live Service

Query A — "Which supplier offers the lowest price for SKU-001?"
```

```
| | Before fix | After fix |
|---|---|---|
| **Tools called** | `query_all_suppliers`, `select_best_supplier` | `query_all_suppliers` (×3, one per
| **Agent answer** | *"The recommended supplier for SKU-001 is PrimeParts Direct with a unit price of $
| **Correct?** | ■ PrimeParts list price is $16.00; the $14.72 figure is the multi-criteria-weighted va

### Query B — "Generate a purchase order for SKU-004 from TechDistributors for 100 units at $25."

| | Before fix | After fix |
|---|---|---|
| **Tools called** | `generate_purchase_order`, `query_all_suppliers`, `select_best_supplier`, `calcula
| **Agent answer** | *"The optimal supplier selected is PrimeParts Direct, with a unit price of $34.96.
| **Respected user intent?** | ■ Replaced supplier (TechDist → PrimeParts), quantity (100 → 129), pri

Query B's "after" response is empty because the output sanitiser in
`guardrails_config.py` redacts the PO file path. That's a separate
... (1 more lines)
```

15. Conclusions

Across the 11 labs, the open-ended deployment + CI/CD task, and Part A of the final exam, the Dynamic Inventory Reorder Agent demonstrates the full lifecycle of an industrial Agentic AI system: from problem framing and domain grounding, through multi-agent orchestration, security hardening, quantitative evaluation, and human-in-the-loop safety controls, to containerised deployment, automated quality gates, and post-deployment drift monitoring with closed-loop improvement evidence.

The most valuable lesson from Part A: a high-frequency feedback loop wired into the live service surfaced two distinct user-intent violations in just twelve interactions. The fix — a one-paragraph system-prompt amendment — produced verifiable, reproducible behaviour change without touching the graph, tool code, or guardrail layer. This is exactly the kind of cheap, evidence-backed iteration that the drift-monitoring infrastructure is designed to enable.

Appendix A — Repository Structure

```
AI-Capstone-Lab/
■■■ PRD.md                          (Lab 1)
■■■ Architecture_Diagram.png        (Lab 1)
■■■ requirements.txt                (pinned dependencies)
■■■ .github/workflows/main.yml      (Lab 10 / Open-Ended)
■■■ Part_A/                         (Labs 2-11 implementation)
■   ■■■ Initial_Data/               5 CSVs + 1 calendar
■   ■■■ ingest_data.py              Lab 2 RAG pipeline
■   ■■■ tools.py                    Lab 3 @tool + Pydantic
■   ■■■ graph.py                    Lab 3 ReAct loop
■   ■■■ multi_agent_graph.py        Lab 4 + agents_config.py
■   ■■■ agent_personas.md           Lab 4 doc
■   ■■■ collaboration_trace.log     Lab 4 evidence
■   ■■■ persistence_test.py         Lab 5
■   ■■■ approval_logic.py           Lab 5 HITL
■   ■■■ guardrails_config.py        Lab 6 rules
■   ■■■ secured_graph.py            Lab 6 graph (patched for Part A)
■   ■■■ security_report.md          Lab 6 adversarial table
■   ■■■ test_dataset.json           Lab 7 — 25 cases
■   ■■■ run_eval.py                 Lab 7 / Lab 10 — CI-ready
■   ■■■ evaluation_report.md        Lab 7 summary
■   ■■■ observability_link.txt      Lab 7 LangSmith
■   ■■■ bottleneck_analysis.txt     Lab 7 analysis
■   ■■■ schema.py                   Lab 8 Pydantic models
■   ■■■ main.py                     Lab 8 FastAPI
■   ■■■ api_test_results.txt        Lab 8 curl output
■   ■■■ Dockerfile                  Lab 9
■   ■■■ .dockerignore               Lab 9
■   ■■■ docker-compose.yaml         Lab 9
■   ■■■ docker_build.log            Lab 9 evidence
■   ■■■ eval_threshold_config.json  Lab 10 legacy
■   ■■■ eval_thresholds.json        Open-Ended versioned thresholds
■   ■■■ ci_breaking_change_demo.py  Lab 10 / Open-Ended
■   ■■■ broken_graph.py             Lab 10 'broken' fixture
■   ■■■ ci_pass_log.txt             Lab 10 passing build
■   ■■■ ci_fail_log.txt             Lab 10 failing build
■   ■■■ app.py                      Lab 11 Streamlit + feedback
■   ■■■ analyze_feedback.py         Lab 11 drift analyser
■   ■■■ drift_report.md             Lab 11 findings
■   ■■■ improved_prompt.txt         Lab 11 prompt v2
■   ■■■ Deployment_Report.md        Open-Ended write-up
■■■ Part_B/                         (MCP exploration, separate from Final Exam Part B)
■■■ Final_Exam/                     (final exam deliverables)
    ■■■ Part_A/                     (this report's Section 14)
    ■   ■■■ seed_feedback.py
    ■   ■■■ feedback_log.json
    ■   ■■■ analyze.py
    ■   ■■■ analysis_report.md
    ■   ■■■ improvement_demo.md
    ■■■ Part_B/                     (covered by Part_B_Report.pdf)
```

Appendix B — Environment & Reproduction

```
# One-shot bootstrap
python -m venv .venv
.venv/Scripts/activate              # Windows
# source .venv/bin/activate         # macOS / Linux
pip install -r requirements.txt
pip install pypdf duckduckgo-search reportlab pygments  # Part B + reports

export GROQ_API_KEY='gsk_...'

# Build the inventory RAG index, then start the services
cd Part_A
python ingest_data.py
python -m uvicorn main:app --host 0.0.0.0 --port 8000 &
python -m streamlit run app.py --server.port 8501 --server.headless true &

# Final Exam Part A
python Final_Exam/Part_A/seed_feedback.py
python Final_Exam/Part_A/analyze.py

# Final Exam Part B
python Final_Exam/Part_B/ingest_data.py
python Final_Exam/Part_B/run_tests.py     # regenerates evaluation_results.md

# Regenerate the two report PDFs
python Final_Exam/gen_reports.py
```